

DIGITAL STACK

Knowledge. Performance. Growth.

PYTHON

Dezvoltarea unor competențe solide de Python pentru AI,
automatizare și fluxuri de
date

DIGITAL STACK

Knowledge. Performance. Growth.

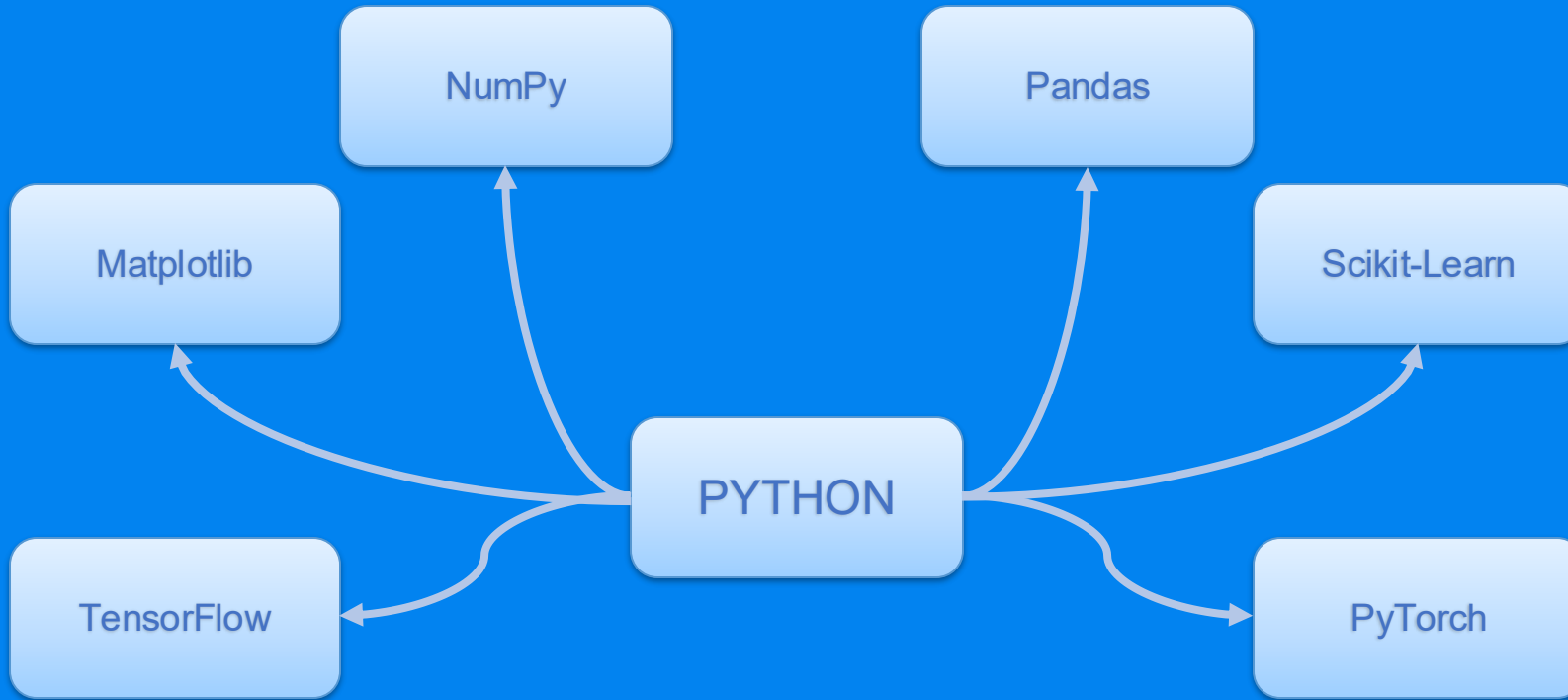
SESIUNEA 1: PYTHON - RECAPITULARE

Recapitularea conceptelor de bază pentru programarea în Python

AGENDĂ

- Python Recap
- Tipuri de Date
- Funcții
- Module
- Medii Virtuale (venv)
- Construirea unui tool CLI simplu

DE CE PYTHON PENTRU AI?



Python este limbajul dominant în AI datorită sintaxei simple și ecosistemului bogat de biblioteci pentru procesare de date, machine learning și deep learning

PYTHON BASICS

Tipuri de Date Fundamentale

```
name = "Alice"  
age = 25  
height = 1.72  
is_student = True
```

- str → text
 - int → numere întregi
 - float → numere reale
 - bool → True/ False
- Tot ceea ce manipulează un program Python este reprezentat prin tipuri de date.

Colecții

list

```
skills = ["Python", "Git", "Docker"]
```

dict

```
user = {  
    "name": "Alice",  
    "age": 25  
}
```

tuple

```
coordinates = (44.43, 26.10)
```

set

```
countries = {"RO", "FR", "UK"}
```

- Ordonate
- Mutabile
- Permite duplicate

- Key - value
- Acces rapid prin cheie

- Ordonate
- Imutabile

- Valori Unice
- Elimină duplicatele

- Alegerea structurii de date potrivite simplifică semnificativ implementarea unei soluții.

Operații Uzuale

```
skills = ["Python", "Git", "Docker"]

user = {
    "name": "Alice",
    "age": 25
}

print(skills[0])
print(user["name"])

skills.append("AI")

print(len(skills))
```

- Accesarea elementelor din listă prin index
- Accesarea valorilor din dicționar prin cheie
- Adăugarea de elemente într-o listă
- Determinarea dimensiunii unei colecții

Funcții

De ce folosim funcții?



```
def greet(name):  
    return f"Hello {name}"  
  
message = greet("Alice")  
print(message)
```

- Reutilizare
- Organizarea codului
- Testare mai ușoară
- Evitarea duplicării

Funcții cu Type Hints

```
def add(a: int, b: int) -> int:  
    return a + b
```

- Cod mai ușor de înțeles
 - Ajutor din partea IDE-ului
 - Detectarea mai rapidă a erorilor
-
- Type hints nu sunt obligatorii în Python, dar sunt recomandate în proiectele moderne.

Module

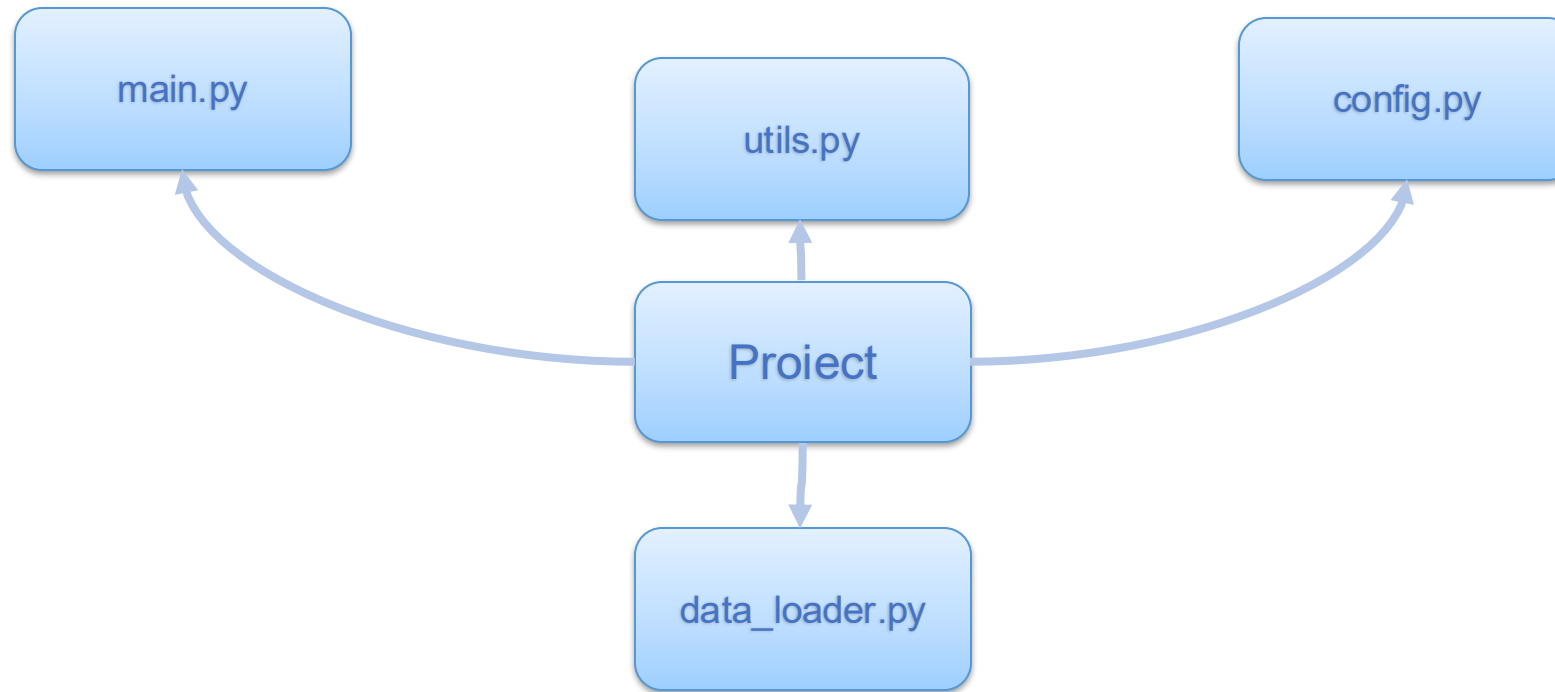
Ce este un modul Python?

- Un fișier .py care conține cod reutilizabil.

Beneficii

- Separarea responsabilităților
- Cod mai ușor de întreținut
- Reutilizare

Structura standard a unui Proiect



- Pe măsură ce un proiect crește, codul trebuie organizat în fișiere și module.

Problema Dependințelor

Scenariu

Project A needs pandas 1.x

Project B needs pandas 2.x

One system Python installation

Problema

Bibliotecile și versiunile pot intra în conflict între proiecte.

Medii Virtuale (Virtual Environments)

```
python -m venv .venv

# Windows
.venv\Scripts\activate

# Linux / Mac
source .venv/bin/activate
```

Ce obținem?

- Dependențe izolate
- Proiecte independente
- Configurație reproductibilă

Instalarea pachetelor

```
pip install requests  
pip list  
pip freeze > requirements.txt
```

- Bibliotecile necesare unui proiect sunt instalate în mediul virtual.

Exercițiul 1 - Primul nostru CLI Tool

Implementați o aplicație Python care primește un nume prin linia de comandă și afișează un mesaj de salut pentru utilizatorul respectiv.

Aplicația trebuie să valideze existența argumentului primit. Dacă utilizatorul nu furnizează niciun argument, trebuie afișat un mesaj care indică utilizarea corectă a aplicației.

Exercițiul 2 - Organizarea codului în funcții și module

Pornind de la soluția exercițiului anterior, reorganizați aplicația astfel încât logica de generare a mesajului de salut să fie mutată într-un modul separat.

Aplicația trebuie să conțină cel puțin o funcție definită de utilizator și cel puțin un import realizat dintr-un fișier Python creat de voi.

Funcționalitatea aplicației trebuie să rămână identică.

Exercițiul 3 - Salutarea utilizatorilor dintr-un fișier

Extindeți aplicația astfel încât aceasta să primească drept intrare un fișier text care conține câte un nume pe fiecare linie.

Pentru fiecare nume găsit în fișier trebuie generat și afișat un mesaj de salut.

Liniile goale trebuie ignorate.

Organizați soluția în funcții cu responsabilități clare și utilizați module separate pentru logica de citire a fișierului și generarea mesajelor.

Exercițiul 4 - Statistici despre utilizatori

Pe baza datelor citite din fișier, implementați o funcționalitate care calculează și afișează statistici despre utilizatorii găsiți.

Aplicația trebuie să determine:

- numărul total de înregistrări;
- numărul de nume distincte;
- numărul de apariții pentru fiecare nume.

Rezultatele trebuie afișate într-un format ușor de citit.

Soluția trebuie să utilizeze liste, dicționare și funcții definite de utilizator.

Provocare (individual)

Adăugați funcționalități suplimentare care să determine:

- numele cu cea mai mare lungime;
- numele cu cea mai mică lungime;
- lungimea medie a numelor;
- clasamentul numelor ordonat descrescător după frecvența aparițiilor.

Încercați să implementați aceste cerințe fără a modifica semnificativ structura aplicației existente.

DIGITAL STACK

Knowledge. Performance. Growth.